KasSigner

Complete User Guide

Air-gapped offline signing device for Kaspa. From unboxing to multisig, backup and recovery: everything the device does, and how to use it.

Release v1.0.6

Date August 2026

Licence GPL-3.0

Repository github.com/InKasWeRust/KasSigner

Contact kassigner@proton.me

You are the sole guardian of your keys. No company, no server, no device stores them for you. Manage your funds with care, verify every transaction on screen, and protect your seed phrase as you would protect your sovereignty.

Don't trust, verify. Sign offline. Broadcast sovereign.

Table of Contents

1. What KasSigner Is (and Is Not)

KasSigner is an open-source offline signing device built on the ESP32-S3. It generates private keys, signs Kaspa transactions via QR code exchange, and never connects to any network. All key material lives in RAM only and is destroyed on power-off.

What KasSigner IS:

- An offline signing device: generates keys, signs transactions, exports via QR
- A seed generator
- A steganographic backup tool: hides encrypted seeds inside JPEG photos
- Stateless: all key material in RAM, destroyed on power-off
- Open source: 100% Rust, reproducible Docker builds

What KasSigner is NOT:

- NOT a hardware wallet: no secure element, no tamper detection, no persistent storage
- NOT professionally audited: it has been through several security reviews, every finding checked against the source, but not a paid engagement with an independent firm
- NOT resistant to physical attacks: voltage glitching, decapping, cold-boot reads of SRAM; and JTAG on any board that has not been eFuse-provisioned
- NOT long-term key storage: your backup IS your storage, not the device

2. Supported Hardware

Why the ESP32-S3?

The ESP32-S3 is a dual-core 240MHz microcontroller with 512KB SRAM, 8MB PSRAM, and built-in USB. It runs bare-metal Rust with no operating system, no network stack, and no background processes. There is nothing between your keys and the silicon. The chip is inexpensive, widely available, and supported by a mature Rust toolchain. Its LCD_CAM peripheral provides native DVP camera input for QR scanning without external ICs. Two I2C buses, SPI, and SDHOST give direct access to display, touch, camera, and SD card, all controlled register-by-register in our code, with no vendor libraries or binary blobs in the signing path.

	Waveshare ESP32-S3-Touch-LCD-2	M5Stack CoreS3 Lite
Price	~$25	~$45
Display	ST7789T3 320x240	ILI9342C 320x240
Camera	OV2640 / OV5640 DVP (auto-detect at boot)	GC0308 QVGA
Touch	CST816D capacitive	FT6336U capacitive
SD Card	SDHOST native (fast)	Bitbang SPI
Audio	none	AW88298 speaker
PMU	none (backlight PWM via LEDC)	AXP2101 + AW9523B
PSRAM	8MB octal	8MB quad

Waveshare pin map:

```
Camera: XCLK=8 PCLK=9 VSYNC=6 HREF=4 D0-D7=12,13,15,11,14,10,7,2
Camera I2C1: SDA=21 SCL=16 PWDN=17
Display SPI: MOSI=38 SCLK=39 CS=45 DC=42 RST=0 BL=1
Touch I2C0: SDA=48 SCL=47 INT=46
SD SDHOST: CLK=39 CMD=38 D0=40 | Battery: ADC=GPIO5
```

M5Stack CoreS3 Lite pin map:

```
Camera: XCLK=2 PCLK=13 VSYNC=42 HREF=41 D0-D7=15,16,48,12,10,40,39,11
Display SPI2: MOSI=37 SCLK=36 CS=3 DC=35
Touch I2C0: SDA=12 SCL=11 INT=21
SD SPI: MOSI=37 SCLK=36 CS=4 (bitbang, shared bus)
PMU: AXP2101+AW9523B I2C0 | Speaker: AW88298 I2S
```

3. Installing the Build Toolchain

Two ways to get a firmware onto a board. Build it locally with the toolchain below, or build it in Docker (chapter 22), which is the path that produces a binary you can verify against the published hashes. Both end with a flash over USB.

Step 1: Install Rust

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
```

Step 2: Install espup (Xtensa toolchain)

```
cargo install espup
espup install
```

Step 3: Install espflash

```
cargo install espflash
```

Step 4: Clone and verify

```
git clone https://github.com/InKasWeRust/KasSigner.git
cd KasSigner/tools
cargo run --bin kassigner-setup
```

> The setup checker verifies Rust, espup, Xtensa toolchain, espflash, Docker, and serial ports. It shows exact install commands for anything missing.

4. Building and Flashing

Step 1: Connect board via USB-C and verify serial port

```
# macOS:
ls /dev/cu.usbmodem*
# Linux:
ls /dev/ttyUSB* /dev/ttyACM*
```

Step 2: Build and flash

Waveshare (drop **ov5640-af** if the plain OV5640 or the OV2640 module is fitted; add **ov2640-wide** for the wide-angle OV2640):

```
cd bootloader
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo run --release --features ov5640-af
```

M5Stack CoreS3 Lite:

```
cd bootloader
cargo run --release --no-default-features --features m5stack
```

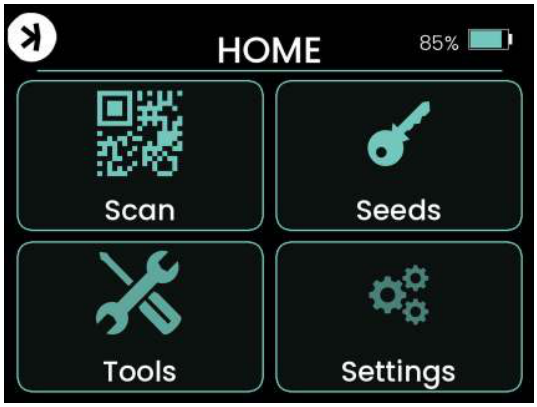
First build takes a few minutes (compiles all dependencies). Subsequent builds take seconds. The device reboots automatically after flashing and shows the home screen.

> At every boot the firmware runs its self-tests and its cryptographic known-answer tests, including 27 consensus sighash vectors, and prints **All crypto KATs passed** on the serial log. This takes about two seconds and cannot be compiled out of a shipped image. Do not build a flash you intend to use with **skip-tests**.

5. The Home Screen

After boot, KasSigner shows a 2x2 grid with four main areas:

- **Scan**: scan QR codes (KSPT, PSKT/PSKB, SeedQR, kpubs and descriptors for multisig)
- **Seeds**: manage seed slots, view addresses, export keys
- **Tools**: create seeds, import keys, BIP85, multisig, stego, message signing
- **Settings**: display brightness, camera tuning (Waveshare) or audio (M5Stack), SD card, about screen



Tap any of the four cards to enter that section. The Kaspa logo and battery indicator are shown on the home screen only. Inside any menu, tap the top-left corner to go back.

6. Creating a New Wallet

From the home screen, tap Tools and then Seed Tools. Three ways to generate a seed, each from a different physical source; the hardware random number generator is not one of them (it feeds nonces and salts, never seeds).

Method A: New Seed (camera, plus IMU on Waveshare)

- Tap **New Seed**, choose 12 or 24 words
- The device grabs eight camera frames and hashes the frame-to-frame sensor noise, mixed with IMU noise on Waveshare, the eFuse MAC, SYSTIMER and timing jitter, via SHA-256
- If the scene is too dark or too still the capture is refused with **Need more light**: point the camera at something lit and moving and try again
- Words shown one at a time, tap to advance

> **WARNING:** Write down every word on paper. These words ARE your wallet. Do not photograph. Do not type into any internet-connected device.

Method B: Dice Seed (verifiable entropy)

- Tap **Dice Seed**, choose 12 or 24, roll a physical die 99 or 198 times
- Tap the dice face (1-6) after each roll. Fully verifiable: the same rolls produce the same mnemonic in any BIP39 tool

Method C: Touch Seed

- Tap **Touch Seed**, choose 12 or 24, then scribble on the screen
- Your finger's positions and timing are the entropy source; the bar fills as events accumulate. A finger held still records nothing, so the bar does not fill and no seed is produced

Calculate Last Word

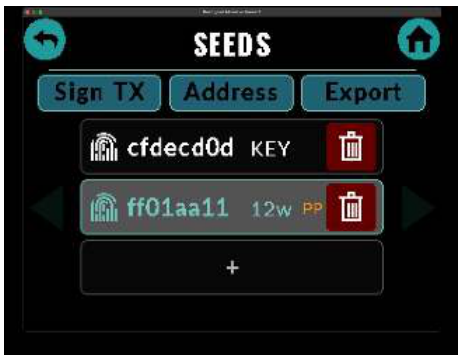
- Tap **Calc Last Word**, enter the first 11 or 23 words, the device computes the valid checksum word

BIP39 Passphrase (25th word)

After seed creation you can add an optional passphrase. This creates a hidden wallet: the 12/24 words alone lead to a decoy wallet, words + passphrase lead to the real wallet. The passphrase is never stored on the device or in any backup the device writes; where you keep it is yours to decide.

7. Seed Management (Multi-Slot)

KasSigner holds up to 16 seed slots in RAM. Nothing is persisted; all slots are lost on power-off.



- Each slot shows a fingerprint hex code, the slot type (KEY, 12w, 24w), and PP if a passphrase is active
- The active slot is highlighted in teal; this is the one used for signing and address display
- Tap any slot to activate it
- Red trash icon securely zeroizes and removes a slot from RAM
- Empty + slot: tap to go to Tools for seed creation or import
- Three action buttons at the top: Sign TX, Address, Export; they operate on the active slot

Slot types:

- **Mnemonic (12w / 24w)**: full capabilities: derive addresses, sign, BIP85, SeedQR, kpub, xprv
- **XPrv**: account-level key: derive addresses, sign, kpub. No BIP85 or SeedQR
- **Raw Key (KEY)**: single address only: sign with one key, no derivation tree

Automatic and manual wipe

- **Idle wipe**: after four minutes without a touch, every loaded seed and derived key is wiped. Reload from your backup to continue
- **Silent wipe**: on the home screen, hold the logo corner for four seconds without lifting. There is no prompt and no progress bar; when the hold completes the device wipes and shows **Wiped**. A quick tap does nothing, so an observer sees nothing

8. Addresses and Keys

Derivation Paths

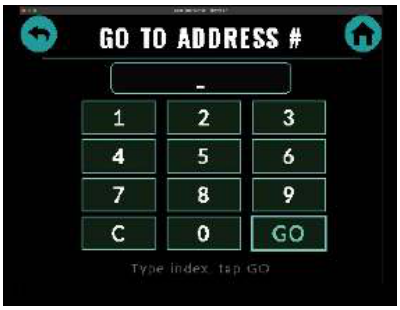
KasSigner uses BIP-44 derivation for Kaspa single-sig and the Kaspa 45' scheme for multisig:

- Receive: **m/44'/111111'/0'/0/N**, addresses you share to receive funds
- Change: **m/44'/111111'/0'/1/N**, internal addresses for transaction change outputs
- Multisig: **m/45'/111111'/0'/<cosigner>/<chain>/N**, see chapter 11

The first 20 receive addresses and 5 change addresses are pre-cached when a seed is loaded. Higher indices are derived on demand.

Viewing Addresses

From the Seeds screen, tap Address to see the receive address for the active seed at index #0.



- Tap the address text to display it as a full-screen QR code
- Use the < and > buttons to navigate between address indices
- Tap the #N button to jump to a specific index using the numeric keypad
- Tap the RECEIVE / CHANGE toggle (center button) to switch between receive and change address views
- Raw Key (KEY) slots show a single address with no navigation; they have no derivation tree

Exporting Keys

From the Seeds screen, tap Export to open the Export menu:

- **Seed Backup**: Show Seed Words, QR Export (CompactSeedQR / Standard SeedQR / Plain Text QR), Backup to SD (encrypted)
- **Watch-Only**: kpub as QR, kpub to SD, kpub Multisig QR
- **Signing Keys**: xprv Account (Show as QR / Encrypt to SD), Private Key
- **Steganography**: hide your seed inside a JPEG photo on SD card

> **Two kpubs, two labels. kpub as QR** is the 44' watch-only key for KasSee or any companion wallet. **kpub Multisig QR** is the 45' key you hand to the other cosigners of a multisig wallet. The two are byte-indistinguishable in form; the label on the device is the only thing that tells them apart, so scan the right one for the job.

The kpub allows viewing all addresses and balances but CANNOT sign transactions. Safe to import into an internet-connected companion wallet for building transactions.

Private Key Export

From the Export menu under Signing Keys, tap Private Key: enter the receive address index (#N), the device derives the private key for m/44'/111111'/0'/0/N and displays it as a hex QR code. Tap anywhere to zeroize the key from memory and return to the menu.

> **WARNING:** The private key gives full control of that single address. Export only to air-gapped devices. Never photograph it or expose it to a networked device.

9. Signing a Transaction (KSPT)

Why KSPT?

Kaspa's native transaction format is PSKT (Partially Signed Kaspa Transaction), a JSON structure with hex-encoded fields. It is universal, and verbose: a simple 1-input, 2-output transaction produces over 2,000 bytes of JSON, several QR frames across an air gap.

KasSigner uses KSPT (KasSigner Packed Transaction), a compact binary format designed for air-gapped QR transport, currently **KSPT v4**. It strips the JSON overhead and encodes values as raw bytes; a transaction that needs several frames in PSKT often fits in one QR. For multisig, KasSigner also reads PSKT and PSKB (Partially Signed Kaspa Bundle) natively, the standard formats for M-of-N signing.

KSPT is not a Kaspa standard; it is a KasSigner format understood by the device and by KasSee, which translates between the network's PSKT and KSPT for QR transport.

Air-gapped signing workflow

The device never connects to any network; data moves by QR or SD card. Load a seed into a slot first; without an active seed, signing fails.

1. KasSee builds an unsigned transaction and displays it as a QR (KSPT for single-sig, PSKT/PSKB for multisig)
2. KasSigner scans the QR, parses the transaction, and shows a review screen
3. You verify every output, the change, and the fee on the device screen
4. The device signs and displays the signed transaction as a QR
5. KasSee scans the signed QR and broadcasts to the Kaspa network

Scan the transaction

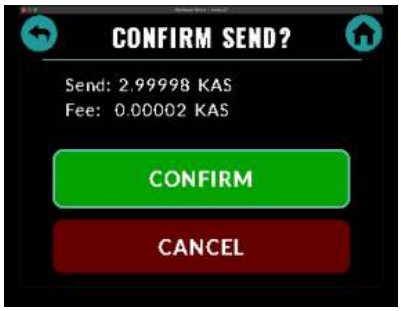
From the home screen, tap Scan. Point the camera at the QR displayed by KasSee. For multi-frame QR codes, hold steady; the device detects and decodes each frame automatically.

Review the transaction

The device shows the total, the fee, the number of inputs and outputs, and then every output with its destination address and amount. Outputs the transaction claims as change or as your own are labelled. Where a payload or a covenant is involved, the payload hash and the covenant id are shown too, so they can be checked against what you expect. For a multisig spend the change label is verified against the loaded descriptor (chapter 11).

Confirm or cancel

Tap CONFIRM to sign or CANCEL to abort.



Signing

The device derives the correct private key for each input (searching both receive and change chains, or following the derivation hint for multisig), computes the sighash using keyed Blake2b-256, and signs with Schnorr.

Export signed transaction

The device displays the signed transaction as a QR code, single or multi-frame; scan it in KasSee to broadcast.

> The first air-gapped hardware-signed Kaspa transaction was broadcast on mainnet on March 30, 2026 using KasSigner. TX ID: 2faa58b247d8bf5b8f8d3d2a467b6821738e083ff7fcad06dfeffb09c520046d

Transaction size limits

The signer accepts up to **32 inputs and 8 outputs** per transaction. If a transaction exceeds that, the device shows **Too many inputs, consolidate UTXOs first** and returns to the menu. Use KasSee's UTXO selection to pick up to 32 UTXOs when sending, or consolidate in batches first. Small UTXOs left behind by many payments are exactly what consolidation is for.

10. Signing Messages

KasSigner can sign arbitrary text messages with any address key. This proves you control an address without making a transaction.

How it works

From the Tools menu, tap Single Signature, then Sign Message.

- Choose **Type manually** to enter text using the on-screen keyboard
- Or choose **Load .TXT from SD** to sign the contents of a text file on SD card

The device hashes the message with SHA-256 and signs the hash with Schnorr using the active address key. The result is displayed as a QR code containing the original message text, the Kaspa address that signed it, and the 64-byte Schnorr signature. Anyone can verify the signature against the address.

The same menu holds **Commit Secret** and **Decrypt Secret**, the device side of the commit-reveal covenant; see the Covenants and Stealth guide.

11. Multisig (M-of-N, 45')

KasSigner implements the Kaspa 45' multisig scheme (written up in **KIP: Multisig Wallet Conventions for Kaspa**, github.com/kaspanet/kips/pull/39). Each cosigner derives at **m/45'/111111'/0'/<cosigner>/<chain>/<index>**, keys are sorted so every implementation computes the same address, and chain 1 is change. M of the N keys must sign to spend.

Step 1: Export your multisig kpub

Seeds, Export, Watch-Only, **kpub Multisig QR**. Each cosigner shows this QR to whoever creates the wallet. It is the 45' key, not the watch-only 44' one (chapter 8).

Step 2: Create the wallet

From the Tools menu, tap Multisig, then Create Multisig. Set M (required signatures) and N (total keys) with the +/- buttons, then NEXT. For each key position choose **Scan kpub QR** (another cosigner's multisig kpub) or **Use Loaded Seed** (a seed already in a slot). Every seed that takes part is a cosigner; the device derives its key at that cosigner's index.

Step 3: Address and descriptor

With all N keys added, the device sorts them and shows the multisig address; tap it for a full-screen QR, or save it to SD. The **Descriptor** screen lists every cosigner key and offers the descriptor as a QR and as a file on SD, plain or encrypted with a password.

The descriptor is a second secret. A seed alone cannot find or spend multisig funds: the addresses depend on every cosigner's key and nothing on chain records the set. Back up **seed and descriptor**. Recovery of a multisig wallet also means scanning one address family per cosigner, not one.

Step 4: Watch and spend from KasSee

In KasSee open Multisig, then **Load a descriptor and one of its addresses**: scan the descriptor QR from the device, and scan or paste one of the wallet's addresses. The address identifies your cosigner branch, so KasSee knows which of the N families is yours. KasSee then shows the wallet, its addresses and UTXOs, and builds spends across several addresses as a PSKB. On the device, tap Scan, review, CONFIRM; the device shows the partially signed bundle as a QR. Hand it to the next cosigner's device (scan, review, sign again) or back to KasSee, which relays it and broadcasts once M signatures are present.

Change verification

When a multisig transaction claims an output as change, the device tries to reproduce that address from a loaded descriptor that already reproduces one of the transaction's inputs:

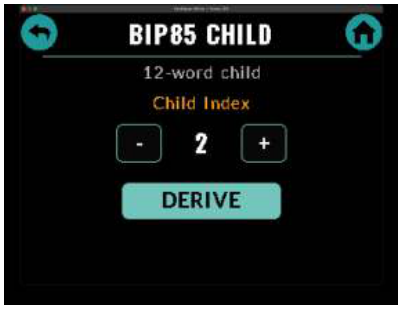
- **Verified** (teal): the descriptor reproduces the address
- **Unverifiable** (orange, with a ?): no trusted descriptor is loaded; treat the output as a payment and check the address yourself
- **FORGED** (red): the descriptor contradicts the claim. The device refuses to sign

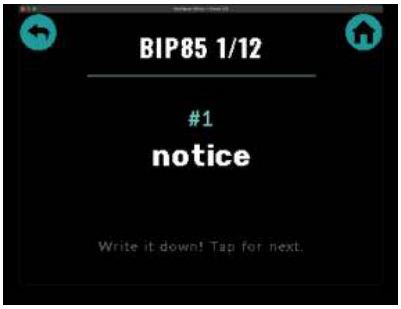
Multisig wallets created before 45' used 44' HD keys (multi_hd descriptors); they still load and spend. New wallets are 45' only.

12. BIP85 Child Mnemonics

BIP85 derives independent child mnemonics from a master seed. Each child is a complete wallet that cannot be linked back to the parent. Use this to create separate wallets for different purposes from one master backup.

From the Tools menu, tap Seed Tools, BIP85 Child. Choose 12 or 24 words. Set the child index (0, 1, 2, ...) using +/- buttons, then tap DERIVE. The child mnemonic is displayed word by word; write each word down, tap for next.





Each index produces a different child. Index 0 always produces the same child from the same master. Write down the child words separately; they are a complete, independent wallet.

13. Backup: Encrypted SD Card

KasSigner encrypts backups with AES-256-GCM and saves them to a MicroSD card. The key is derived from a password you choose via PBKDF2-HMAC-SHA256 (100,000 rounds) with a random salt and nonce per file. Seeds, xprv account keys, raw keys, multisig descriptors and unsigned transactions all use the same container, with the purpose bound into the key so one kind of file cannot be passed off as another.

Creating a backup

- Insert a FAT32-formatted MicroSD card
- From the Seeds screen, tap Export, Seed Backup, Backup to SD
- Enter a password on the keyboard and tap OK
- The device encrypts and writes the backup file (progress bar shown)
- File is named with the seed fingerprint (e.g. SDC0B1)

Restoring from backup

- From the Tools menu, tap Import / Export, Import from SD, and choose the file type
- The device lists the matching files; files matching a loaded seed show a label (e.g. "Seed #2")
- Tap a file, enter the password, and the seed is decrypted and loaded
- Progress bar: 0-80% PBKDF2 derivation, 90% decrypt, 100% loaded

Deleting a backup file

- Tap the red trash icon on a file card, confirmation screen
- CANCEL (teal outline, left) or DELETE (red, right)
- Tap DELETE, the button changes to "HOLD 4s", press and hold 4 seconds to confirm
- Release early or move your finger off the button to cancel

WARNING: If you forget the password, the backup cannot be recovered. There is no reset mechanism. Keep your password safe.

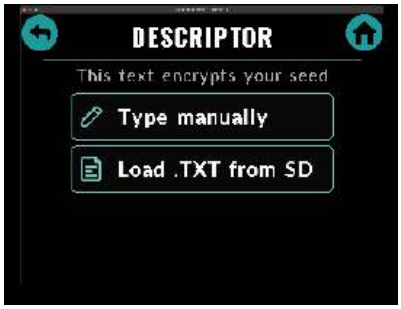
14. Backup: Steganographic JPEG

KasSigner's steganographic backup hides your encrypted seed inside an ordinary JPEG photograph. Three layers:

- **Layer 1, which file?** The encrypted seed rides inside a photo that looks like every other one. Two carriers: **Descriptor** puts it in the EXIF metadata and survives re-saving but dies to metadata stripping; **Picture** puts it in the image's own compressed data, survives stripping and dies to re-compression. Run the export twice for both
- **Layer 2, which password?** AES-256-GCM, key derived by PBKDF2 from the image description you enter, which is stored in the photo as a normal caption. It IS the password
- **Layer 3, which passphrase?** Even if decrypted, the words alone lead to a decoy wallet. The real wallet needs the BIP39 passphrase, which is never stored on the device or in the photo

Creating a stego backup

Seeds, Export, Steganography. Select a JPEG from the SD card, then enter or load the image descriptor:



Optionally add a recovery hint for your passphrase; the hint is stored with the seed and the answer to it IS your passphrase. Confirm the overwrite: the original JPEG is modified in place.

Run both exports on the same photo, in either order. Type the descriptor exactly the same way for both: import asks for it once and tries both carriers with it, so a single different character means one of the two payloads never decrypts. Re-running one export replaces that carrier's previous payload; a photo carries at most one Descriptor payload and one Picture payload, and a second seed needs a second photo.

Restoring from stego backup

From the Tools menu, tap Import / Export, Stego Import. Choose the photo and enter the exact image descriptor used at backup; the device tries both carriers and reports which one carried the seed. If a hint was stored, the device shows it; the answer is your passphrase, enter it as the 25th word.

> **WARNING:** The descriptor must match exactly, including capitalization, spaces and punctuation. There is no recovery mechanism if forgotten. Test every backup: import it back on the device before trusting it. For the design and its limits read docs/STEGANOGRAPHY.md.

15. Backup: SeedQR (Paper)

SeedQR encodes your BIP39 mnemonic as a QR code that you print on paper or engrave on metal. It is a community-standard format compatible with SeedSigner, Krux, and other air-gapped devices. KasSigner supports three variants:

- **CompactSeedQR**: binary encoding of BIP39 word indices. Smallest QR, fastest scan. Recommended
- **Standard SeedQR**: 4-digit zero-padded BIP39 indices. Compatible with SeedSigner
- **Plain Words QR**: mnemonic words as plain text. Human-readable but largest QR

Exporting a SeedQR

Seeds, Export, Seed Backup, QR Export. The device displays the QR on screen; tap for the grid view to hand-copy the pattern onto paper.

Importing a SeedQR

From the home screen, tap Scan. Point the camera at any SeedQR (compact, standard, or plain). The device auto-detects the format, decodes the mnemonic, and loads it into a seed slot.

> **WARNING:** SeedQR is unencrypted; anyone who sees or photographs the QR has your seed. Treat SeedQR backups with the same care as written seed words. SeedQR stores the mnemonic only, not your passphrase.

16. Importing Seeds and Keys

Import via QR scan

From the home screen, tap Scan. The device auto-detects the QR type:

- CompactSeedQR / Standard SeedQR / Plain Words QR: mnemonic
- xprv QR: account-level extended private key
- kpub QR: watch-only or multisig extended public key
- Multisig descriptor QR
- KSPT, PSKT, PSKB: transactions to sign

Import words manually

Tools, Seed Tools, Import Words. Enter each BIP39 word using the on-screen keyboard with autocomplete. After 12 or 24 words the device validates the checksum and loads the seed.

Import from SD card

Tools, Import / Export, Import from SD, then choose the file type: **Seed Backup**, **Transaction**, **kpub (Watch-Only)**, **Multisig Address**, **Multisig Descriptor**, **Covenant Restore**. See chapter 13 for the encrypted ones.

Import from steganographic JPEG

Tools, Import / Export, Stego Import. See chapter 14.

Import raw private key

Tools, Import / Export, Import Raw Key. Enter a 32-byte hex private key via QR scan or the keyboard. This creates a single-address KEY slot: no derivation tree, one address, one key.

> After any mnemonic import you are prompted for an optional BIP39 passphrase (25th word). Raw keys and xprv imports skip this step.

17. SD Card Management

KasSigner uses MicroSD cards for encrypted backups, steganographic JPEGs, multisig addresses and descriptors, transactions, covenant restore data, and text files. From Settings, SD Card you can check the card status, format it, or test read/write.

Requirements

- A current **microSDHC** card, 4 to 32 GB, from a known brand. That is what the device is tested with, it comes FAT32 out of the box, and any speed class is fine; data volumes are tiny
- FAT32 only; exFAT and NTFS are not supported. Larger SDXC cards can work if you reformat them FAT32, but that is not the tested path
- Very old cards (SD v1, under 2 GB) are accepted by the driver but not recommended; they are the ones most likely to fail on a low-voltage SPI or SDHOST bus

File types on SD

- **SDxxxx**: encrypted seed backup, named by fingerprint, no extension
- **XPxxxx**: encrypted xprv account key
- **.JPG**: JPEG photos for stego backup
- **.TXT**: text files for message signing or descriptor loading
- Multisig address and descriptor files, plain or encrypted
- Transaction files (unsigned KSPT, encrypted) and covenant restore files

Troubleshooting

- SD not detected: remove and reinsert. Ensure FAT32 format. Try a different card
- File not showing: filenames must be 8.3 format (uppercase, no long names)
- Write error: check the card is not write-protected (physical lock tab on adapter)

> The SD card is never required for basic operation. Seeds can be created, imported, and used for signing without one.

18. Settings

From the home screen, tap Settings.

Display

Adjusts the LCD backlight level. Higher brightness improves QR scanning by the companion camera but draws more power.

Camera (Waveshare only)

Live viewfinder with sensor tuning controls (exposure, gain and related registers) for the OV5640 and OV2640, so QR scanning can be adjusted to your light. The M5Stack GC0308 works at its defaults and has no such screen.

Audio (M5Stack only)

Confirmation beeps and volume. The Waveshare board has no speaker.

SD Card

Card status (detected, type, capacity), Format FAT32, and Test R/W. See chapter 17.

About

Firmware version, code-segment hash, hardware target, and the eFuse Secure Boot state of the unit.

19. Camera and QR Tips

KasSigner decodes QR codes using rqrr_nostd, a no_std Rust fork of the rqrr 0.10.1 library, bounded on device to QR versions V1 through V13, all four ECC levels, with full Reed-Solomon error correction and perspective transform. A single successful decode is accepted, with no voting or multi-pass heuristics.

Camera hardware

The Waveshare board supports two sensors with runtime auto-detection, the OV2640 (wide-angle, 2MP) and OV5640 (5MP), captured 480x480 through DMA into PSRAM. The M5Stack CoreS3 Lite uses a GC0308 QVGA sensor with grayscale capture at 320x240.

Lighting

- Bright, even lighting works best. Avoid shadows across the QR
- The screen displaying the QR should be at maximum brightness
- Avoid direct sunlight on the lens

Distance and angle

- Hold the device 10 to 15 cm from the QR
- Straight on works, but a slight tilt of 20 to 45 degrees often scans faster: it takes the screen's own reflection out of the lens and gives the decoder a cleaner image
- Dense codes (multi-frame transactions, descriptors) want the camera closer, so the modules stay large in the image; small codes (V1, V2, a SeedQR) scan better a little further back

Multi-frame QR codes

- KasSee displays animated frames that cycle automatically; hold steady and the device assembles them
- Progress dots below the viewfinder: gray pending, teal received
- If a frame is missed keep holding; the animation loops and the device catches up
- Up to 64 frames per transfer

The OV5640 runs warm during long continuous capture and image quality may drop a little after several minutes; it recovers after a rest. If the camera seems stuck, unplug USB, wait 10 seconds, and reconnect.

20. Key Derivation Architecture

KasSigner follows the BIP32/BIP39/BIP44 hierarchy used by Kaspa wallets for single-sig, and the 45' scheme for multisig. Understanding both trees matters for verifying that your addresses match between KasSigner and other software.

Single-sig path

```
m / purpose' / coin_type' / account' / change / index
m / 44'      / 111111'   / 0'       / 0      / 0
```

- purpose = 44' (BIP44)
- coin_type = 111111' (Kaspa)
- account = 0' (hardened)
- change = 0 receive chain, 1 change chain
- index = 0, 1, 2, ...

Multisig path

```
m / 45' / 111111' / 0' / cosigner / chain / index
```

Every cosigner shares the same account and each takes its own cosigner index; keys are sorted before the script is built so any implementation computes the same address. Chain 1 is change.

Key levels

- **Master key (m)**: from the BIP39 seed. Only exists with the full mnemonic
- **Account key (m/44'/111111'/0')**: the xprv level. Derives all addresses and signs, cannot derive BIP85 children
- **Address key (m/.../0/N)**: one secp256k1 keypair
- **kpub**: the extended public key at account level, derives public addresses, cannot sign. This is what KasSee imports

When a transaction does not spend a whole UTXO, the remainder goes to a change address on chain 1. KasSigner searches both chains when signing, so it correctly signs inputs from either.

21. Security Model

KasSigner's security rests on air-gap isolation and ephemeral key storage. It is not a hardware wallet with a secure element; it is a signing device that makes deliberate tradeoffs. The full model, layer by layer, is in SECURITY.md and the Security Architecture document; this is the short version.

What KasSigner protects against

- **Network attacks**: no WiFi, no Bluetooth, no radio. The radios are never initialised and their clocks and power domain are switched off at boot
- **Malware on your computer**: keys never cross USB. Data moves only by QR and SD card
- **Memory persistence**: all key material lives in SRAM only, wiped after use, on idle, on panic and on power-off
- **Supply chain (with Secure Boot)**: only firmware signed with the burned key runs, and the firmware checks its own hash and signature and its cryptographic primitives against published vectors at every boot

What KasSigner does NOT protect against

- **Physical access attacks**: voltage glitching, decapping, cold-boot reads of SRAM; JTAG on an unprovisioned board. No tamper detection
- **Side channels**: no constant-time guarantee on every path (comparisons and BIP32 reduction are constant-time). Power analysis and EM attacks are possible
- **Evil maid**: a swapped device is not detectable unless Secure Boot is enabled and you verify the firmware hash
- **Firmware bugs**: several security reviews, not a professional audit

Defense in depth

- Air gap, ephemeral RAM, Secure Boot V2, software firmware verification, Rust memory safety, encrypted backups, steganographic hiding, BIP39 passphrase, reproducible builds, boot-time known-answer tests

Panic wipe

If the firmware hits an unrecoverable error, it wipes SRAM with volatile writes before halting, so no key material survives a crash even if the device is powered and captured.

> Your last line of defence is the BIP39 passphrase: never stored by the device or in any backup it writes. Keep it apart from the words.

22. Reproducible Builds (Docker)

Anyone can build the firmware from source and verify that the binary matches the release, bit for bit.

Step 1: Build the toolchain image (once)

```
docker build --platform linux/amd64 -f Dockerfile.base -t kassigner-toolchain:v3 .
```

Step 2: Build the firmware

```
docker build --platform linux/amd64 -t kassigner-build .
```

Step 3: Verify

```
docker run --rm kassigner-build
```

Compare the SHA-256 hashes with the unsigned hashes published with the release. Signed and unsigned images differ, because the signature is embedded; a verifier has no signing key, so compare unsigned against unsigned.

Why it works

- Forced x86_64: identical hashes on Intel and Apple Silicon
- Pinned toolchain: base image by content digest, packages and compilers by version
- Cargo.lock: every dependency pinned
- Hash convergence: up to five build passes, the last two must agree, or the build fails
- SOURCE_DATE_EPOCH and objcopy: no timestamps, no build paths in the binary

Full procedure and troubleshooting in docs/REPRODUCIBLE_BUILD.md. Don't trust, verify.

23. Secure Boot (eFuse)

> **ALL eFuse operations are IRREVERSIBLE.** A mistake can permanently brick the board. Read docs/EFUSE_RUNBOOK.md in full before touching any command, and test on a sacrificial board first.

A provisioned KasSigner has two independent signature checks: hardware Secure Boot V2 (RSA-3072, verified by the silicon ROM against a key digest burned into eFuse) and the software Schnorr check in the firmware itself. Secure Boot is the root of trust; the software check is defense in depth under it. Both boards, Waveshare and M5Stack CoreS3 Lite, take the same chip-level procedure.

What the runbook covers

- Reading the eFuse state of a fresh board and identifying it
- Generating and backing up the RSA-3072 key, burning its digest, revoking unused key slots
- Signing the bootloader and the app, flashing signed images BEFORE enabling Secure Boot
- Enabling Secure Boot, closing hardware and USB JTAG
- The two fuses that must never be burned (DIS_USB_SERIAL_JTAG and DIS_DOWNLOAD_MODE), why, and what was verified on hardware

Flashing a provisioned device afterwards uses the download-mode sequence in docs/BUILD_FLASH_GUIDE.md: unplug USB, hold BOOT, plug in, release; only signed images will run. The About screen shows the unit's Secure Boot state.

24. Live Display Mirror

KasSigner can mirror its display to a computer over the USB Serial/JTAG interface, useful for demos, screenshots and troubleshooting. This is a development feature: it needs a dev build with the **mirror** feature (production builds gate USB after boot and cannot be built with it). You need two terminals.

Terminal 1: flash and run the firmware

```
cd bootloader
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo run --release --features mirror,skip-tests
```

Terminal 2: launch the viewer

```
cd tools
cargo run --release --bin kassigner-mirror -- /dev/cu.usbmodem21201
```

Replace the port with your device's. A 960x720 window opens showing the device screen in real time (~15 fps, minifb, pure Rust); arrow keys navigate; **--export** writes PNGs.

> The mirror transmits pixel data only, never key material. Do not use a mirror build for real funds.

25. KasSee: Watch-Only Companion

KasSee is a browser-based watch-only companion wallet, pure Rust compiled to WebAssembly, no install, no server. It holds only your extended public key and can see addresses and balances but cannot sign. It is also the project's test bench for new device features; KasSigner works with any wallet that speaks PSKB and QR.

Getting started

- Open kassigner.org in any modern browser; installable as a PWA on mobile
- Import your kpub by scanning the QR from KasSigner or pasting it
- KasSee connects to a Kaspa node over WebSocket to fetch balances and UTXOs

Features

- Address derivation, receive and change, with funded (orange) and used (red) badges and explorer links
- Build transactions with fee levels; Send Max; manual UTXO selection up to 32 inputs; UTXO consolidation in batches
- Broadcast: scan the signed QR from KasSigner
- Multisig: import a 45' descriptor, watch the wallet, build spends across addresses, relay between cosigners, broadcast
- Covenants++ and stealth payments: see the Covenants and Stealth guide
- Tokens: KRC-20 balances, KRC-721 NFTs, KNS domains (read-only)
- Custom node in Settings; own node recommended for privacy

Signing workflow

1. In KasSee, enter the destination and amount, tap Send
2. KasSee shows the unsigned transaction as an animated QR
3. On KasSigner, tap Scan and point the camera at it
4. Review and confirm on the device
5. KasSigner shows the signed transaction as a QR
6. In KasSee, tap Broadcast and scan it

> KasSee runs in your browser, an untrusted environment. It never has your keys, but it can be tricked into showing you the wrong thing. The device screen is the only trusted display: verify there before you confirm.

26. Troubleshooting

Device won't boot / blank screen

- Unplug USB, wait 10 seconds, reconnect
- Try a different USB cable; some are charge-only
- Hold BOOT while plugging in to enter download mode, then reflash

Camera frozen / no image

- Power cycle (unplug 10 seconds)
- Check lighting
- Waveshare auto-detects OV2640/OV5640 at boot; if detection fails, power cycle
- M5Stack GC0308 is lower resolution: move the QR closer

QR not scanning

- Increase brightness on the screen showing the QR
- 10 to 15 cm, parallel
- Multi-frame: hold steady and let all frames cycle
- Avoid reflections; tilt slightly

SD card not detected

- FAT32 only; cards over 32GB need manual FAT32 formatting
- Reinsert firmly
- Settings, SD Card, Format FAT32
- Try another card

Signing failed

- Make sure the correct seed is loaded and active (fingerprint)
- The inputs must be spendable by the active seed (or, for multisig, by a cosigner it holds)
- "Too many inputs": the limit is 32 inputs and 8 outputs; select or consolidate in KasSee

Wrong password on SD import

- Passwords are case-sensitive; re-enter carefully

Touch not responding

- Taps fire on release, not on press; tap and release cleanly
- A firm, deliberate tap is best; avoid wet fingers or screen protectors

Build errors

- Run cargo run --bin kassigner-setup from tools/
- Feature flags: default (plus ov5640-af or ov2640-wide) for Waveshare, --no-default-features --features m5stack for M5Stack

LCD ghost image

The ST7789T3 retains charge from bright pixels; the firmware applies a grey wash before camera init. If ghosting persists, power cycle.

27. Command Reference

Firmware build commands

```
# Waveshare (OV5640 with AF module):
cd bootloader
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo run --release --features ov5640-af

# Waveshare (OV2640 wide-angle):
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo run --release --features ov2640-wide

# M5Stack CoreS3 Lite:
cargo run --release --no-default-features --features m5stack
```

Clippy (the three project configurations, all with -D warnings)

```
cargo clippy --release --no-default-features --features m5stack -- -D warnings
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo clippy --release --features ov5640-af -- -D warnings
cargo clippy --release --no-default-features --features m5stack,e12-capture,rng-probe -- -D warnings
```

Docker reproducible builds

```
docker build --platform linux/amd64 -f Dockerfile.base -t kassigner-toolchain:v3 .
docker build --platform linux/amd64 -t kassigner-build .
docker run --rm kassigner-build
```

One-step installer (macOS)

```
bash Install.sh
```

Mirror viewer (dev build, separate terminal)

```
cd tools && cargo run --release --bin kassigner-mirror -- /dev/cu.usbmodem21201
```

Environment check

```
cd tools && cargo run --bin kassigner-setup && cd ..
```

KasSee Web (WASM build)

```
cd kassee && RUSTUP_TOOLCHAIN=stable ./build.sh
```

This compiles the Rust source to WebAssembly and writes the WASM and JS bindings to kassee/web/pkg/. Open kassee/web/index.html in a browser to use KasSee locally.